Pattern Matching

Concepts of Programming Languages

CAS CS 320

Outline

- Discuss **pattern matching**
- This means giving a syntax, typing, and semantics
- This also means defining **patterns**, and describing what matching actually means

demo

Pattern Matching

Pattern Matching (High Level)

```
let rec sum l =
  match l with
  | [] -> 0
  | x :: xs -> x + sum xs
```

Values come in different "shapes"

We want to do things based on the different shapes

Matching allows us to condition on the shape of the value

```
sum (1 :: 2 :: [])
```

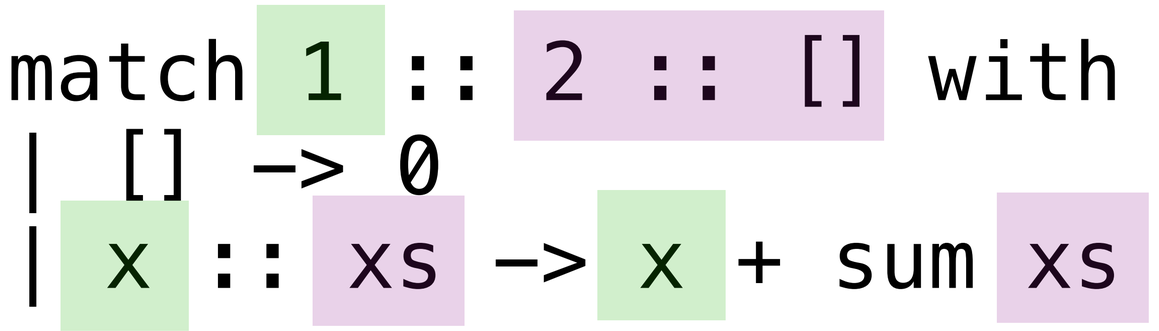
```
match 1 :: 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

```
match 1 :: 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

failed match

```
match 1 :: 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

successful match



create new bindings

```
1 + sum (2 :: [])
```

compute branch with new bindings

```
 1 +
match 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

rinse and repeat

```
1 +
match 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

failed match

```
1 +
match 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

successful match

```
1 +
match 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

create new bindings

```
1 + (2 + sum [])
```

compute branch with new bindings

```
1 + (2 +
match [] with
| [] -> 0
| x :: xs -> x + sum xs)
```

rinse and repeat

```
1 + (2 +
match [] with
| [] -> 0
| x :: xs -> x + sum xs)
```

successful match

```
1 + (2 +
match [] with
| [] -> 0
| x :: xs -> x + sum xs)
```

no new bindings

```
1 + (2 + 0)
```

compute branch

3

fin

Making Things Formal

```
let rec sum l =
  match l with
  | [] -> 0
  | x :: xs -> x + sum xs
```

What do match-expressions look like? **(syntax)**

What is a valid match-expression? **(typing)**

How do match-expressions behave? **(semantics)**

Syntax

Match Syntax

Match Syntax (Example)

```
match 2 + 4 with
| 2 + 4 -> 2 * 4
| _ -> 0
```

Can we do this in OCaml?

Match Syntax (Again)

Patterns (High Level)

```
match x with
| 1 -> e1
| 2 -> e2
| _ -> e3
```

A pattern describes the "shape" of a **value**

But we don't *just* care about the "shape" we also want to **do stuff** with the pieces of the data

Patterns are syntactic representations of values with names

Shallow vs. Deep Pattern Matching

```
let third l =
  match l with
  | _ :: _ :: x :: _ -> Some x
  | _ -> None
```

We're interested implementing *deep* pattern matching, which allows for *nested* patterns

We can also implement *shallow* pattern matching ADTs (which are sometimes called *eliminators*)

Shallow Pattern Matching for Lists

```
<expr> ::= match <expr> with
           | [] -> <expr>
           | <var> :: <var> -> <expr>
```

We can hardcode the possible patterns

This works, but is tedious

Shallow Pattern Matching for Lists

```
let third l =
  match l with
  | [] -> None
  | _ :: l -> (
      match l with
      | [] -> None
      | _ :: l -> (
          match l with
          | [] -> None
          | x :: _ -> Some x
        )
    )
```

Wildcards

```
let third l =
  match l with
  | _ :: _ :: x :: _ -> Some x
  | _ -> None
```

Like OCaml, we'll treat wildcards as distinct from variables

Pattern Syntax

Match Syntax (Again, Again)

Typing

Questions

What makes a valid match expression?

When should I get a type error?

Match Typing

Pattern Typing (High Level)

```
match 1 :: 2 :: 3 with
| [] -> 0
| true -> 2
| x :: _ -> x
```

type error

Pattern Typing (High Level)

```
match 1 :: 2 :: 3 with
| [] -> 0
| true -> 2
| x :: _ -> x
```

type error

Patterns are *typed*. We can check for "reasonableness" of patterns

Pattern Typing (High Level)

```
match 1 :: 2 :: 3 with
| [] -> 0
| true -> 2
| x :: _ -> x
```

type error

Patterns are *typed*. We can check for "reasonableness" of patterns

(In fact, we can check if we have exactly enough patterns, this is *exhaustiveness checking*)

Pattern Typing (High Level)

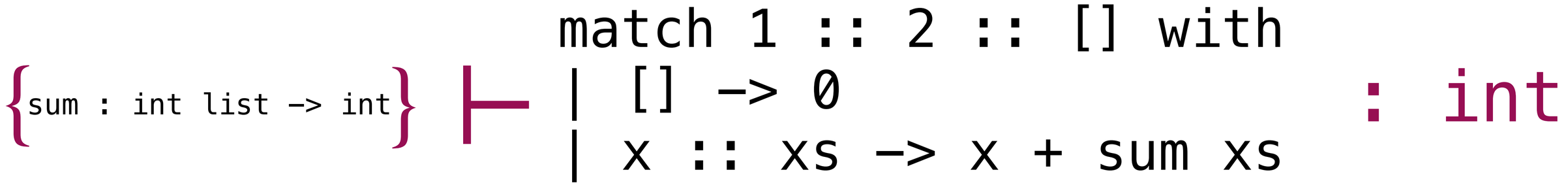
```
match 1 :: 2 :: 3 with
| [] -> 0
| true -> 2
| x :: _ -> x
```

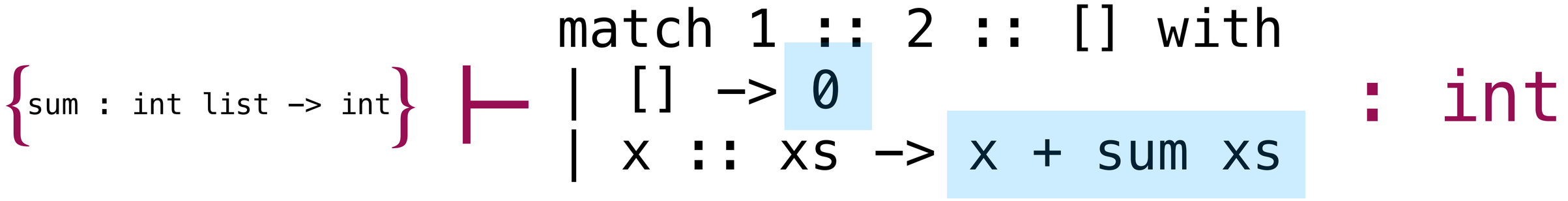
type error

Patterns are *typed*. We can check for "reasonableness" of patterns

(In fact, we can check if we have exactly enough patterns, this is *exhaustiveness checking*)

Key point: patterns *generate* variable declarations


{sum : int list -> int} ⊢
match 1 :: 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
: int



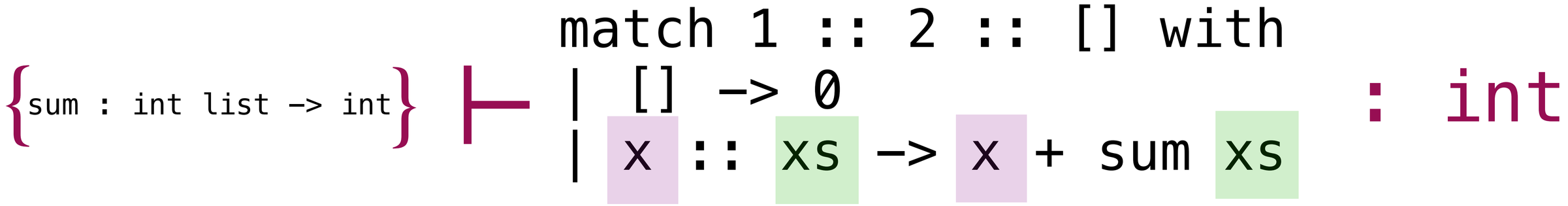
check both branches are ints

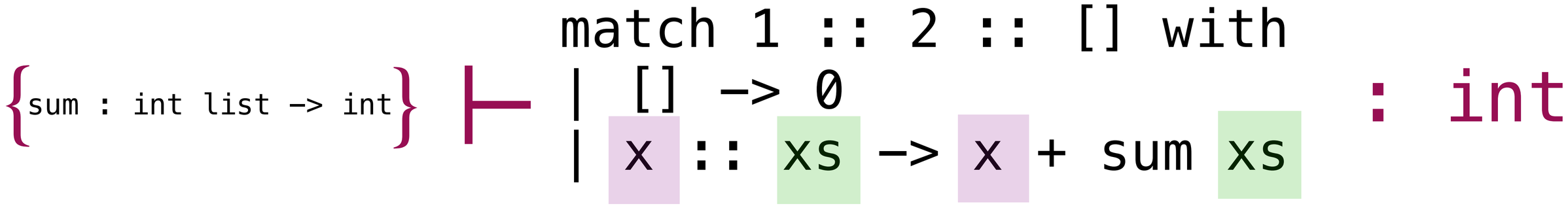
$$\{\texttt{sum : int list -> int}\} \vdash 0 : \texttt{int}$$


{sum : int list -> int} ⊢ x + sum xs : int



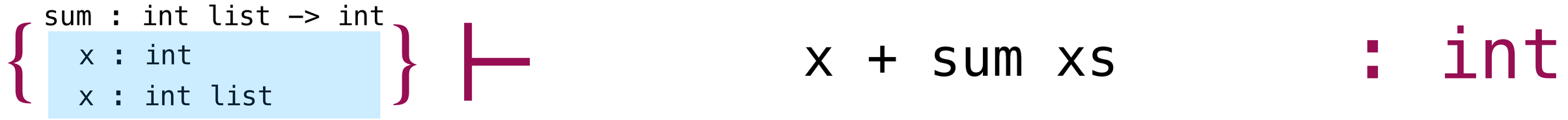
We don't know what x and xs are


{sum : int list -> int}
⊢
match 1 :: 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
: int



the pattern generates new declarations

$$\left\{\begin{array}{l}\texttt{sum : int list -> int}\\ \texttt{\ \ x : int}\\ \texttt{\ \ x : int list}\end{array}\right\} \vdash \texttt{x + sum xs} : \texttt{int}$$



How do we generate these declarations?

Pattern Typing Judgments

$$p : \tau \Rightarrow \Gamma$$

"The pattern p is of type τ and generates the variable declarations Γ"

(Our derivations now have multiple kinds of judgments)

Pattern Typing

Pattern Typing (Example)

Match Typing (Again)

Match Typing (Example)

Semantics

Questions

What does "matching" mean?

How do we find a match?

What do we do when we find a match?

What if we don't find a match?

Matching Semantics

Pattern Matching (High Level)

```
let rec sum l =
  match l with
  | [] -> 0
  | x :: xs -> x + sum xs
```

Pattern Matching (High Level)

```
let rec sum l =
  match l with
  | [] -> 0
  | x :: xs -> x + sum xs
```

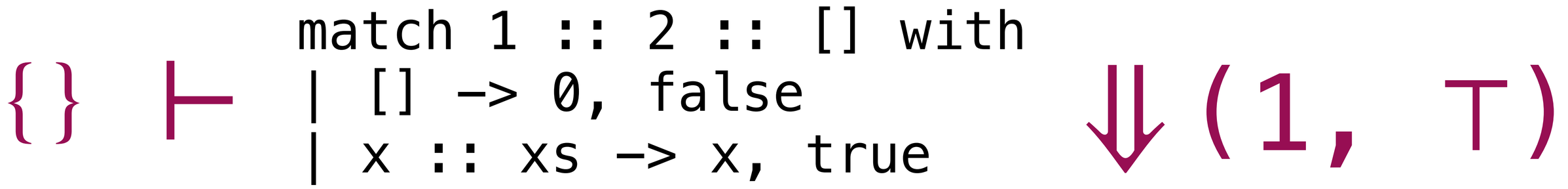
Since patterns "look" like values, we just need to describe when two things "look alike"

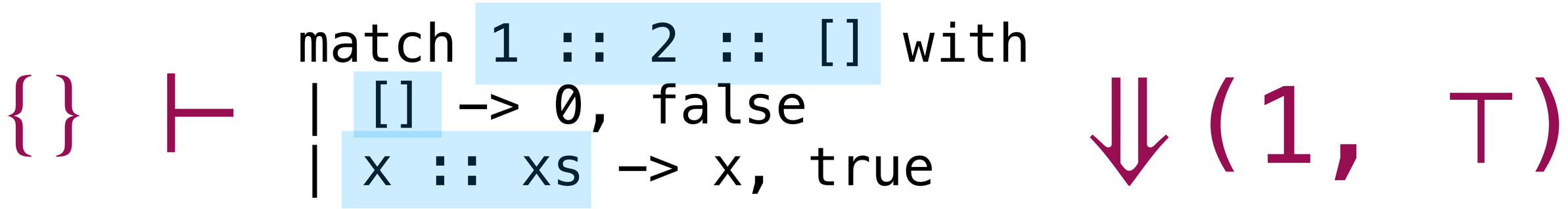
Pattern Matching (High Level)

```
let rec sum l =
  match l with
  | [] -> 0
  | x :: xs -> x + sum xs
```

Since patterns "look" like values, we just need to describe when two things "look alike"

Reminder: matching generates variable bindings


{} ⊢
match 1 :: 2 :: [] with
| [] -> 0, false
| x :: xs -> x, true
⇓(1, ⊤)



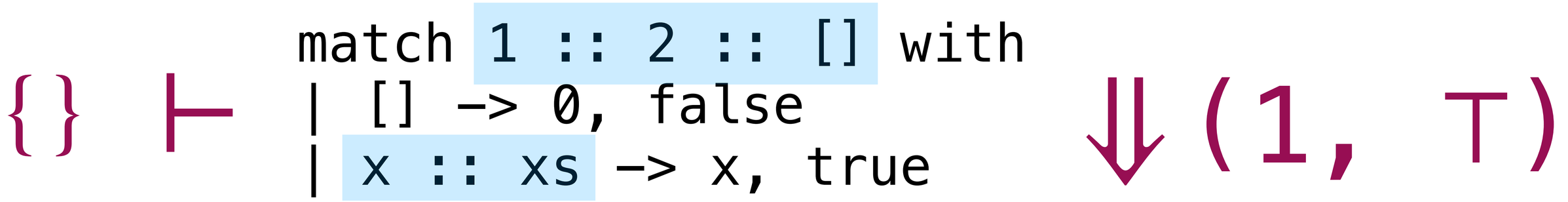
find a branch that the value of the expression matches

$\{\} \vdash$

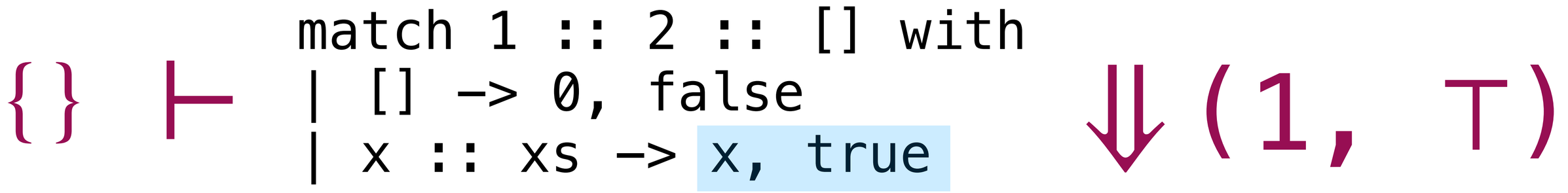
```
match 1 :: 2 :: [] with
| [] -> 0, false
| x :: xs -> x, true
```

$\Downarrow (1, \top)$

failed pattern match

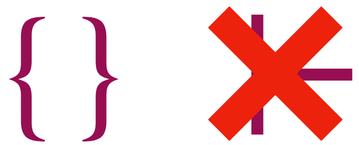


successful pattern match



evaluate success branch

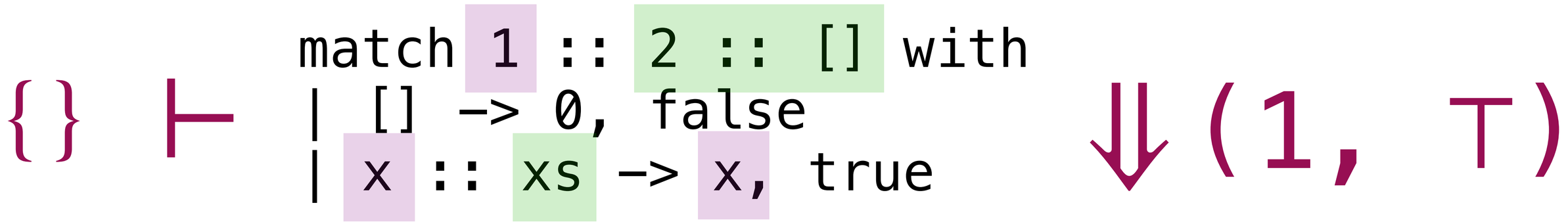
$$\{\} \vdash \texttt{x, true} \Downarrow (1, \top)$$



x, true



We don't know what x is



the pattern generates new bindings

$$\{\texttt{x} \mapsto \texttt{1}, \texttt{xs} \mapsto \texttt{[2; 3]}\} \vdash \texttt{x, true} \Downarrow (1, \top)$$

$$\{\texttt{x} \mapsto 1, \texttt{xs} \mapsto [2; 3]\} \vdash \texttt{x, true} \Downarrow (1, \top)$$

How do we generate these declarations?

Pattern Matching Judgment

$$v \text{ matches } p \Rightarrow \mathscr{E}$$

"The value v matches the pattern p and generates the bindings $\mathscr{E}$"

Pattern Matching

Pattern Matching (Example)

Match Semantics

Match Semantics (Example)

Summary

We can formally define the syntax, typing and semantics of **match expressions**

Deep pattern matching requires the introduction of **patterns**